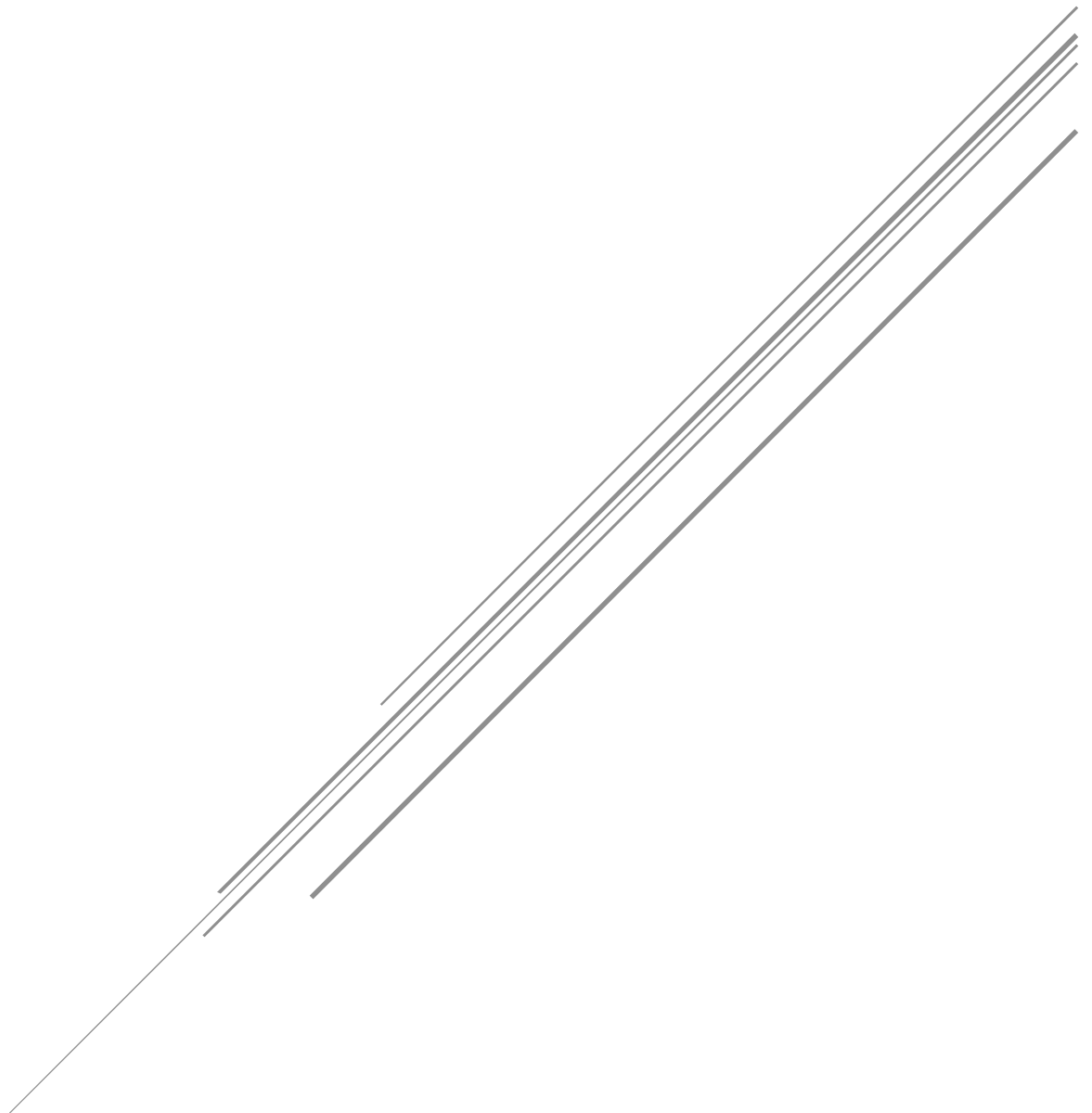


[ORACLE SQL]

[Quick Guide]



Author: Non-Tech to TECH

Table of Contents

Quick Note.....	2
ORACLE SQL	2
Installation of ORACLE APEX.....	2
Import Data in ORACLE APEX.....	3
ORACLE Practical	4
1. Retrieving data from SELECT Clause.....	4
2. Retrieving data from the SELECT + WHERE Clause.....	5
3. Retrieving data from the SELECT + WHERE + AND Clause	5
4. SELECT + WHERE + NOT (!).....	6
5. SELECT + WHERE + OR	6
6. IN, NOT IN, BETWEEN, IS NULL	6
7. And with ()	7
8. Like.....	7
9. SQL Aliases.....	8
10. SQL Concat (Double Pipe -)	8
11. SQL ORDER BY.....	8
12. Single ROW function (SRFs)	9
13. Single ROW function (SRFs) with WHERE Clause.....	10
14. INITCAP, LENGTH, SUBSTR, PAD, TRIM.....	10
15. NUMERIC SRFs (ROUND, TRUNC)	11
16. DATE SRFs (Sysimestamp, add_months, Trunc)	12
17. DATE SRFs - 2 (to_char, to_date)	12
18. DATE SRFs - 3 (LAST_DAY, Next_day).....	13
19. NULL (NVL)	13
20. NULLIF (arg1, arg2).....	14
21. GROUPING FUNCTIONS.....	14
22. GROUPBY.....	15
23. HAVING.....	15
24. SQL SEQUENCE.....	16
25. SELECT within SELECT (Nested / Sub queries)	16
26. JOINS	16
27. INNER JOIN	16
28. LEFT JOIN/LEFT OUTER JOIN.....	16
29. RIGHT JOIN/RIGHT OUTER JOIN.....	17

30.	FULL OUTER JOIN	17
31.	EXISTS / NOT EXISTS.....	17
32.	SELF JOINS.....	17
33.	CROSS JOINS.....	17
34.	NATURAL JOINS	17
35.	USING.....	17
36.	EQUIJOIN	17
37.	CASE WHEN.....	18
38.	OVER	18
39.	PARTITION BY (Continuation of Group)	18
40.	Create TABLE	18
41.	Insert Data into TABLE.....	18
42.	Insert ALL Data into the TABLE	18
43.	CREATE TABLE with Primary KEY	18
44.	INSERT ALL (Conditional Insert into Multiple Tables)	18
45.	INSERT ALL with Condition.....	19
46.	Describe Table	19
47.	ALTER TABLE	19
48.	RENAME COLUMN.....	19
49.	CREATE table with SELECT	19
50.	ADDING NEW COLUMN.....	20
51.	UPDATE VALUE to a COLUMN	20
52.	MODIFY COLUMN	20
53.	MERGE STATEMENT	20
54.	SEQUENCE STATEMENT	20
55.	DELETE TABLE	21
56.	ALTER, DROP TABLE	21
57.	TRUNCATE.....	21
58.	CREATE INDEX	21
59.	ROW ID	21
60.	ALL TABLES.....	21
61.	ALL TABLES/COLUMNS.....	21
62.	ALL OBJECTS.....	21
63.	SYNONYM	22
64.	ROWNUM.....	22
65.	CREATE VIEW (Named Query)	22

66.	TO VIEW the OBJECT	22
67.	TO VIEW the OBJECT based on SCHEMA OWNERS	22
68.	CREATE or REPLACE VIEW	22
69.	FETCH	22
70.	COMMIT, SAVEPOINT & ROLLBACK	22
71.	CASCADE CONSTRAINTS	23
	JOIN / Follow to Explore	23

Quick Note

This E-book will cover the basics of ORACLE SQL. I would request you to go through the execution steps and snaps for better understanding. All the snaps and live steps were captured from Apex. The steps to install & create a workspace has been added for your quick workaround. I wish you all the very BEST & hope this E-book will add value to your professional growth.

"Do understand the limitations of the practical copy considering the access, version & data".

ORACLE SQL

Structured Query Language (SQL) is the set of statements with which all programs and users access data in an Oracle database. Application programs and Oracle tools often allow users access to the database without using SQL directly, but these applications in turn must use SQL when executing the user's request. This chapter provides background information on SQL as used by most database systems. Simple SQL lets you access and manipulate databases.

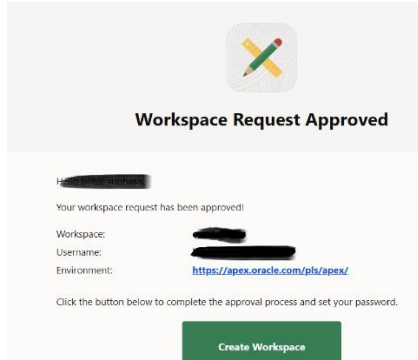
Installation of ORACLE APEX

1. Go to <https://apex.oracle.com/en/learn/getting-started/> and click on **Request for a free workspace**.
2. Fill in the details and the link for the workspace will be shared in your mail ID.

The screenshot shows the 'Request Workspace' form on the Oracle APEX website. It contains the following elements:

- Input fields for 'First name', 'Last name', and 'Email'.
- A text input field for 'Workspace name' with a note: 'The workspace name uniquely identifies your development environment and is used when signing in.'
- A dropdown menu for 'Your Location'.
- Two radio button questions:
 - 'Are you new to Oracle APEX?' with options 'Yes' and 'No'.
 - 'Do you plan to use APEX for education or training?' with options 'Yes' and 'No'.
- A checkbox for 'I agree to the terms of the Oracle APEX Service Agreement'.
- A 'Request Workspace' button at the bottom.

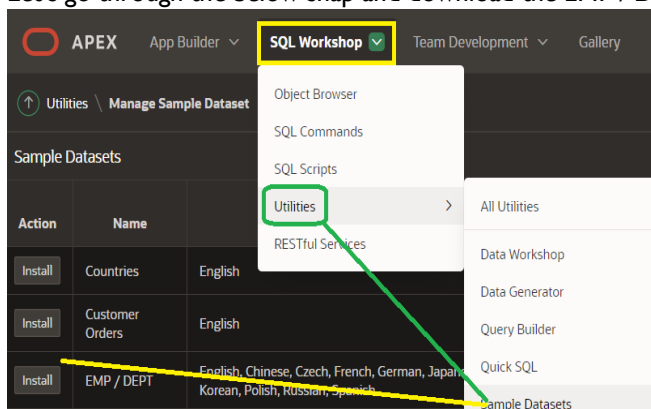
3. Do remember your login and workspace credentials.



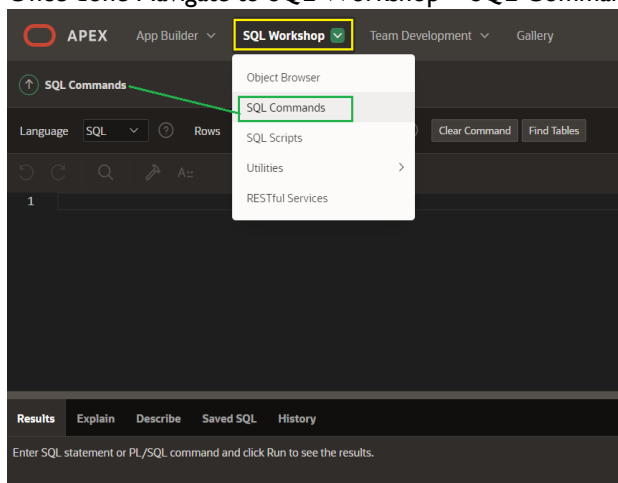
4. Click on the create workspace and now you are ready to roll n roll.
5. Do change the password for the first instance and keep it in your bookmarks for quick access.
6. Remember the workspace and password.

Import Data in ORACLE APEX

1. Hope by this you can see the home page of Apex.
2. We will download the base data to practice our codes.
3. Let's go through the below snap and download the EMP / DEPT table.



4. Once done Navigate to SQL Workshop > SQL Commands (Here we will spend most of our time)



Few Points to remember before jumping into practical

- Oracle is not case-sensitive.

- Oracle is case sensitive while you are looking for a text in single quotes like 'King'.
- Remember to use " (Double Quotes) while looking / conditioning for text output.
- If you want to run a portion of the query: Highlight the code and click the run button.
- * stands for all the columns of the table.
- Recommend all of you go through the link and understand the basics of DB.
- This is a very basic E-book, and I hope you understand the Operators.
- Knowingly in a few places I don't follow the SQL standards of writing the code just to prove my 1st point.
- There would be a few task/Q for self-explore, try to complete those on your own if not reach out to me for the answers.
- You can copy the codes and run them in Apex in case of any doubt.
- Codes are written in purple font color. Questions are in red font color.
- Let's move to the practical section and start digging.
- Do like, share, and don't miss to read through the JOIN/FOLLOW section.

"The great aim of education is not knowledge but action."—Herbert Spencer

ORACLE Practical

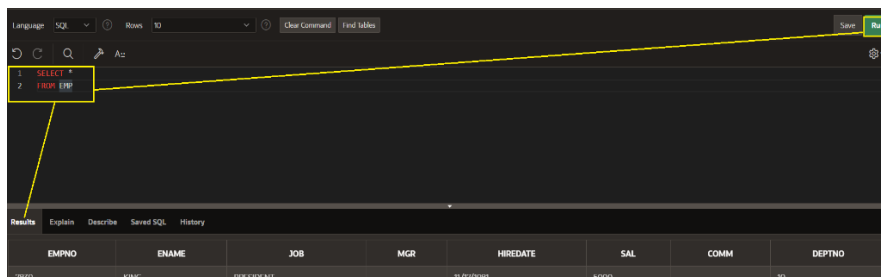
I. Retrieving data from SELECT Clause

A. Select * from EMP --- *SELECT <column> FROM <table>*

Here we are retrieving all the columns from the EMP table.

This is not case-sensitive, write it as you wish and click the RUN button.

It is always good practice to follow certain standards while writing SQL code.



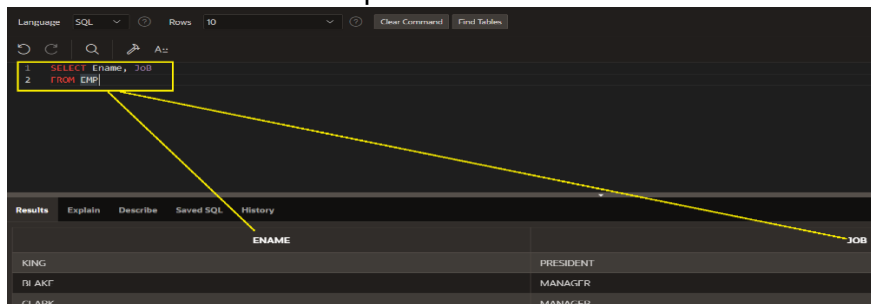
B. *SELECT* Ename, Job FROM EMP

Here we are retrieving selected columns like ENAME & JOB.

Pay attention to the column names (This is not a good practice but just to make you understand)

Q: Derive DEPTNO & DNAME from the DEPT table. (Hint: We have installed two tables EMP & DEPT)

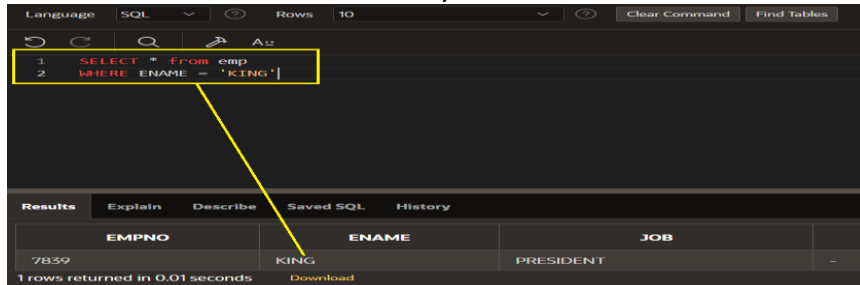
Look for the DEPT table & explore.



2. Retrieving data from the SELECT + WHERE Clause

A. SELECT * from emp WHERE ENAME = 'KING'

Here we are retrieving all the columns in the emp table but looking for the individual name 'KING' Where the condition is useful when you have a condition to follow.



The screenshot shows the SQL Developer interface. The SQL command window contains the query: `1 SELECT * from emp` and `2 WHERE ENAME = 'KING'`. The Results tab is active, displaying a single row of data for the employee KING. A yellow box highlights the query text, and a yellow arrow points from it to the result row.

EMPNO	ENAME	JOB
7839	KING	PRESIDENT

1 rows returned in 0.01 seconds

Task: Change the ENAME from KING to King or KInG and run the code.

Q: Derive all columns for MANAGER (JOB) professional.

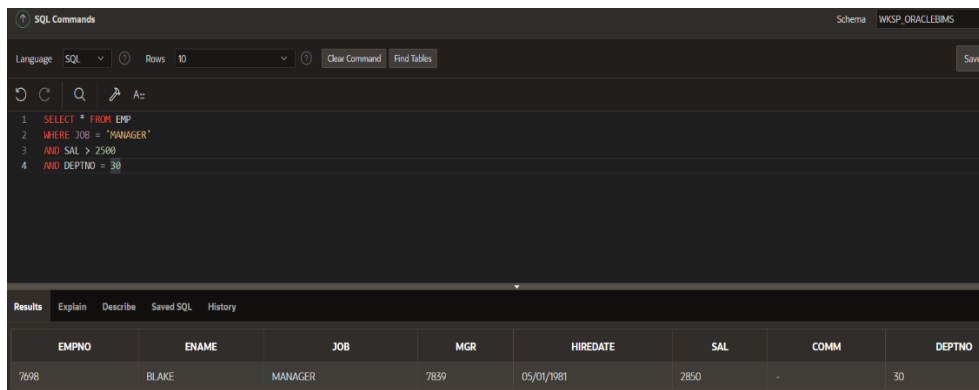
3. Retrieving data from the SELECT + WHERE + AND Clause

A. SELECT * FROM EMP WHERE ENAME = 'FORD' AND SAL = 3000

To satisfy more than one argument we use AND to differentiate those. Remember you can use multiple AND clauses with operators.

The result will appear when all arguments are TRUE.

B. SELECT * FROM EMP WHERE JOB = 'MANAGER' AND SAL > 2500 AND DEPTNO = 30



The screenshot shows the SQL Developer interface. The SQL command window contains the query: `1 SELECT * FROM EMP`, `2 WHERE JOB = 'MANAGER'`, `3 AND SAL > 2500`, and `4 AND DEPTNO = 30`. The Results tab is active, displaying a single row of data for the employee BLAKE.

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7698	BLAKE	MANAGER	7839	05/01/1981	2850	-	30

4. SELECT + WHERE + NOT (!)

A. SELECT * FROM EMP WHERE JOB != 'MANAGER' AND SAL != 3000

Here we are looking for records where the individuals are not Managers and their salary is $\neq$ 3000. Use! When you want to use the NOT clause.

Try $\neq$ and explore the result.

The screenshot shows a SQL interface with a command window and a results table. The command window contains the following SQL query:

```

1 SELECT * FROM EMP
2 WHERE JOB != 'MANAGER'
3 AND SAL >= 3000

```

The results table displays the following data:

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL
7859	KING	PRESIDENT	-	11/12/1981	5000
7369	SMITH	CLERK	7902	12/12/1980	800
7520	ALLEN	SALESMAN	7698	02/21/1981	1600

Q: Derive Manager ENAME belongs to DEPTNO 30. (Hint: In place of * use the column names you req)

Q: Get the Managers a Salary greater than or equal to 2500. (Hint: Use >=)

5. SELECT + WHERE + OR

A. SELECT * FROM EMP where JOB ='MANAGER' OR JOB = 'SALESMAN'

When you want any one statement to be true then use OR command. AND / OR logic are similar to Excel logic.

Run the command on yourself and explore.

Replace the OR with AND. Explore further if you got any ERROR / NO data found.

Don't you feel multiple AND / OR statement referring to the same column is like duplication of effort?

Let's cease that in the coming slide.

6. IN, NOT IN, BETWEEN, IS NULL

A. SELECT ename, hiredate FROM EMP WHERE DEPTNO IN (10,30)

Instead of writing multiple OR statements, use the IN keyword to group them.

As per the above example, we are looking for the NAME & HIRE Date of individuals belonging to DEPT 10 or 30.

B. SELECT * FROM EMP WHERE ENAME NOT IN ('KING', 'CLARK', 'SCOTT')

USE the NOT keyword before IN to exclude the list.

C. SELECT ENAME FROM EMP WHERE HIREDATE BETWEEN '05/01/1981' and '12/03/1981'

To look for records between certain periods and numbers use the BETWEEN clause.

D. SELECT ENAME FROM EMP WHERE MGR IS NULL

The NULL Keyword is useful while looking for a NULL value from a list. Use IS NULL.


```
1 SELECT ename, hiredate FROM EMP
2 WHERE DEPTNO IN (10,30)
3
4 SELECT * FROM EMP
5 WHERE ENAME NOT IN ('KING', 'CLARK', 'SCOTT')
6
7 SELECT ENAME FROM EMP
8 WHERE HIREDATE BETWEEN '05/01/1981' and '12/03/1981'
9
10 SELECT ENAME FROM EMP
11 WHERE MGR IS NULL
```

Q: Highlight Records where the SAL is between 1000 & 2000.

Q: Derive ENAME where COMM is not null. (Hint: Understand the difference between NULL and 0)

Q: Derive ENAME where COMM is 0.

7. And with ()

- A. `SELECT * FROM EMP WHERE JOB = 'SALESMAN' AND (COMM = 300 OR COMM > 1000)`

As per the above example, we are looking for SALESMAN whose commission is either equal to 300 or greater than 1000. To achieve both arguments in one go this need to cover with (). Here the condition evaluates together.

```
1 SELECT * FROM EMP
2 WHERE JOB = 'SALESMAN'
3 AND (COMM = 300 OR COMM > 1000)
```

8. Like

- A. `SELECT * FROM EMP WHERE JOB Like 'SA%'`
B. `SELECT * FROM EMP WHERE JOB Like '%AN'`

```
1 SELECT * FROM EMP
2 WHERE JOB Like 'SA%'
3
4 SELECT * FROM EMP
5 WHERE JOB Like '%AN'
```

Like keyword is useful and being used like a wild card search. If you know the first few letters and looking for the rest remember to keep the % symbol at the end similar to point A. Point B is an example of when you are aware of the last few words.

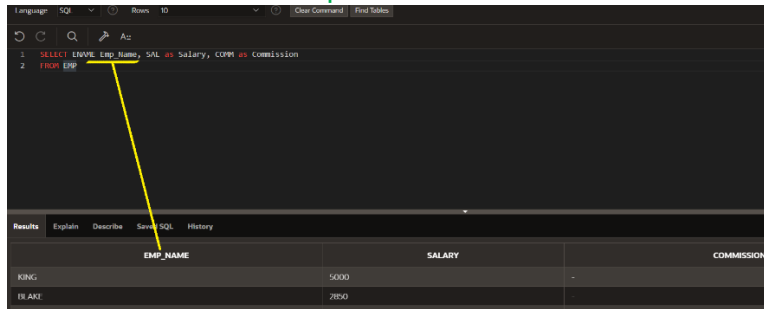
9. SQL Aliases

- A. `SELECT ENAME Emp_Name, SAL as Salary, COMM as Commission FROM EMP`

SQL aliases are used to give a table, or a column in a table, a temporary name.

Aliases are often used to make column names more readable. An alias is created with the AS keyword (not mandatory but a good practice).

Task: Remove the _ from Emp_Name as run the query. Did you get any Errors? Understand the Error code and fix it with the help of "".



The screenshot shows a SQL query window with the following text:

```
1 SELECT ENAME Emp_Name, SAL as Salary, COMM as Commission
2 FROM EMP
```

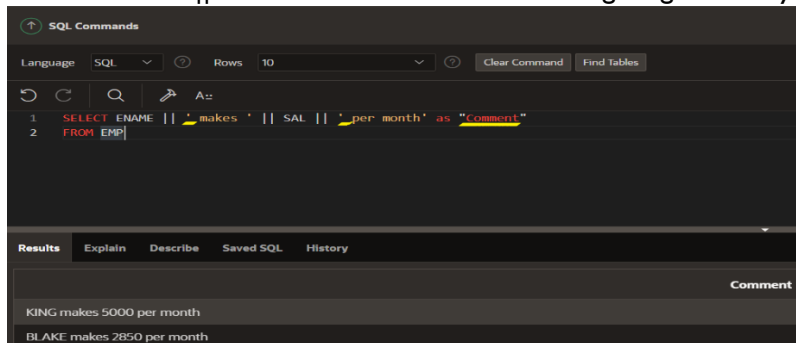
A yellow arrow points from the underlined `Emp_Name` in the query to the `EMP_NAME` column header in the results table below.

EMP_NAME	SALARY	COMMISSION
KING	5000	-
BLAKE	2850	-

10. SQL Concat (Double Pipe - ||)

- A. `SELECT ENAME || ' makes ' || SAL || ' per month' as "Comment" FROM EMP`

The CONCAT || function adds two or more strings together. Pay attention to the space, " & "".



The screenshot shows a SQL query window with the following text:

```
1 SELECT ENAME || ' makes ' || SAL || ' per month' as "Comment"
2 FROM EMP
```

The results table below shows the output of the query:

Comment
KING makes 5000 per month
BLAKE makes 2850 per month

11. SQL ORDER BY

- A. `SELECT * FROM EMP ORDER BY SAL`
B. `SELECT * FROM EMP ORDER BY SAL DESC`

Data is sorted from smaller to largest by default. Use the keyword DESC to show the largest value at the top.

Use the keyword ASC for the default view.

```
SQL Commands
Language SQL Rows 10
1 SELECT * FROM EMP
2 ORDER BY SAL
3
4 SELECT * FROM EMP
5 ORDER BY SAL DESC
```

Q: Get the employee's details for DEPT NO 10 and show the SALARY from Highest to Lowest.
Bring only Name & Salary details – Replace the current names with an alias.

C. `SELECT ENAME, SAL,DEPTNO FROM EMP ORDER BY DEPTNO, SAL`

We can do multiple sorts similar to the above statement but remember it will first execute the first column followed by the next.

```
1 SELECT ENAME,SAL,DEPTNO FROM EMP
2 ORDER BY DEPTNO, SAL
```

ENAME	SAL	DEPTNO
MILLER	1300	10
CLARK	2450	10
KING	5000	10
SMITH	800	20
ADAMS	1100	20
JONES	2975	20
SCOTT	3000	20

12. Single ROW function (SRFs)

In this session, we will go through all the ORACLE functions which will result in a single ROW.
The result will go through each row to execute.
Execute the below queries in APEX.

A. `SELECT CONCAT ('My name is ', ENAME) AS NAME FROM EMP`

The concat function will go through every row and execute the result.

B. `SELECT Upper (ENAME) FROM EMP`

Converting upper value from ENAME.

```

1 SELECT CONCAT ('My name is ', ENAME) AS NAME
2 FROM EMP
3
4 SELECT Upper (ENAME)
5 FROM EMP

```

C. SELECT * FROM DUAL

Creating a dummy table to practice the SRFs instead of using the EMP / DEPT Table.

D. SELECT CONCAT (lower(ENAME), ' is the Name') as NAME from EMP

Combining an argument with a function, we can pass multiple arguments here. Remember concat function will take only two arguments.

Q: Extract the details by using the above statement for DEPT NO 20.

```

1 SELECT * FROM DUAL
2
3 SELECT CONCAT (lower(ENAME), ' is the Name') as NAME
4 from EMP

```

NAME
king is the Name
blake is the Name
clark is the Name

13. Single ROW function (SRFs) with WHERE Clause

A. SELECT * FROM EMP WHERE JOB = Upper('MANAGER')

A single ROW function is wrapped with a where clause. Though this can be simply written as JOB = 'MANAGER'

14. INITCAP, LENGTH, SUBSTR, PAD, TRIM

A. SELECT INITCAP ('My name IS blmAL') as Name FROM DUAL

The function will capitalize the initial letter.

B. SELECT length ('INDIA') as Country FROM DUAL

To count the number of characters between “.”.

Task: Give space post letter A of INDIA and run the query, understand the result.

Q: Check the length of the ENAME in the EMP table.

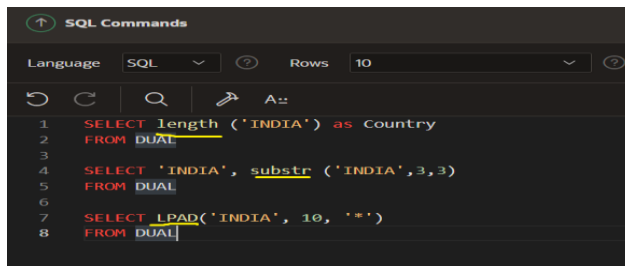
C. SELECT 'INDIA', substr ('INDIA',3,3) FROM DUAL

This function is extracted and has three arguments. 1st argument is actual string, 2nd – Starting position, 3rd – how many characters you want to extract.

D. `SELECT LPAD('INDIA', 10, '*') FROM DUAL`

LPAD function expands the width of a string by adding context and appeared as 3rd argument to the left of the string.

Task: Use RPAD in place of LPAD and explore the outcome.



```
1 SELECT length('INDIA') as Country
2 FROM DUAL
3
4 SELECT 'INDIA', substr('INDIA',3,3)
5 FROM DUAL
6
7 SELECT LPAD('INDIA', 10, '*')
8 FROM DUAL
```

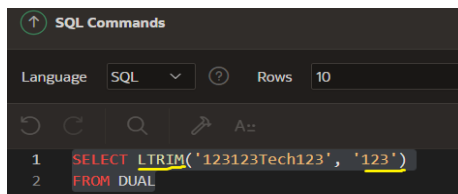
E. `SELECT LTRIM('123123Tech123', '123') FROM DUAL`

LTRIM function removes all specified characters from the left-hand side of a string.

2nd Optional. The string that will be removed from the left-hand side of string1. If this parameter is omitted, the LTRIM function will remove all leading spaces from string1.

Replace LTRIM with RTRIM to perform the function from the right side.

Q: Bring Tech from Tech123123 with help of RTRIM



```
1 SELECT LTRIM('123123Tech123', '123')
2 FROM DUAL
```

15. NUMERIC SRFs (ROUND, TRUNC)

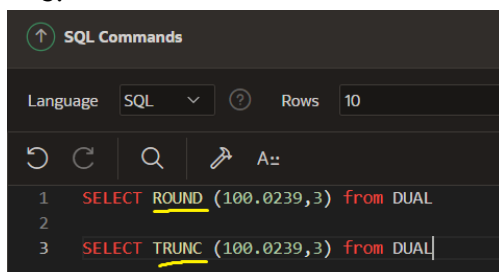
A. `SELECT ROUND(100.0239,3) from DUAL`

Rounding the values based on the position. Above example rounds to the 3rd position. Remember when the digit is ≥ 5 it will round to the next number.

If there is no decimal number defined it will return the exact value. In above case 100.

B. `SELECT ROUND(100.0239,3) from DUAL`

Opposite of the ROUND option where the values are trunked based on the argument even though it's ≥ 5 .



```
1 SELECT ROUND(100.0239,3) from DUAL
2
3 SELECT TRUNC(100.0239,3) from DUAL
```

16. DATE SRFs (Systimestamp, add_months, Trunc)

A. SELECT systimestamp from dual

Self-explanatory.

B. SELECT add_months ('3/4/2023',3) from dual

Add months as the word describes it will jump to future months.

Task: Use – before 3 and see the result. Are we gone backward?

Q: Move 3 months back based on your system date.

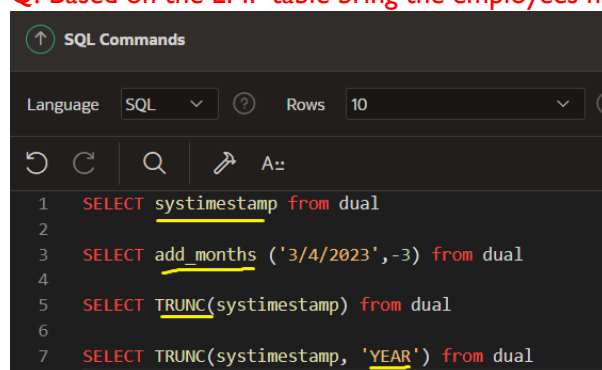
C. SELECT TRUNC(systimestamp) from dual

TRUNC works with the date function as well.

This can be additionally trunked to year or month. Ref Attached.

Q: Bring the start day of the YEAR based on the current sys time.

Q: Based on the EMP table bring the employees hired in 1981.



```
1 SELECT systimestamp from dual
2
3 SELECT add_months ('3/4/2023',-3) from dual
4
5 SELECT TRUNC(systimestamp) from dual
6
7 SELECT TRUNC(systimestamp, 'YEAR') from dual
8
```

Note: SRFs are allowed with a where clause.

17. DATE SRFs - 2 (to_char, to_date)

A. SELECT to_char(systimestamp,'dd-mm-yyyy') from dual

to_char is useful when you are looking to change a default format either in data/string.

Parameter	Explanation
YEAR	Year, spelled out
YYYY	4-digit year
YYY	Last 3, 2, or 1 digit(s) of the year.
YY	
Y	
IYY	
IY	Last 3, 2, or 1 digit(s) of ISO year.
I	
IYYY	
Q	
MM	Month (01-12; JAN = 01).
MON	Abbreviated name of month.
MONTH	Name of month, padded with blanks to length of 9 characters.
RM	Roman numeral month (I-XII; JAN = I).
WW	Week of year (1-53) where week 1 starts on the first day of the year and continues to the seventh day of the year.
W	Week of month (1-5) where week 1 starts on the first day of the month and ends on the seventh.
IW	Week of year (1-52 or 1-53) based on the ISO standard.
D	Day of week (1-7).
DAY	Name of day.
DD	Day of month (1-31).
DDD	Day of year (1-366).
DY	Abbreviated name of day.

Parameter	Explanation
J	Julian day; the number of days since January 1, 4712 BC.
HH	Hour of day (1-12).
HH12	Hour of day (1-12).
HH24	Hour of day (0-23).
MI	Minute (0-59).
SS	Second (0-59).
SSSSS	Seconds past midnight (0-86399).
FF	Fractional seconds.

B. `SELECT to_date('2023-03-04', 'yyyy-mm-dd') from DUAL`

Converting a string into an actual data type. Which can be later merged with `add_months` to jump from the current month to the future/past.

```

1 SELECT to_char(systimestamp, 'dd-mm-yyyy') from dual
2
3 SELECT to_date('2023-03-04', 'yyyy-mm-dd') from DUAL

```

18. DATE SRFs - 3 (`LAST_DAY`, `Next_day`)

A. `select LAST_DAY ('3/4/2023') from dual`

Looking for the last day of the selected month.

B. `Select Next_day ('3/4/2023', 'wed') from dual`

Looking for the upcoming calendar day of a day. Like coming Wednesday which is **HOLI** in INDIA 😊

```

1 select LAST_DAY ('3/4/2023') from dual
2
3 Select Next_day ('3/4/2023', 'wed') from dual

```

19. NULL (`NVL`)

A. `SELECT ENAME, SAL, nvl (COMM,0) as Comm FROM EMP WHERE EMPNO IN (7839,7698, 7654 )`

`NVL` function helps to convert the null values with 0.

`NVL` function works with integers only.

Q: Replace null values with “No Commission”. Hint: Use `to_char` to convert the number into a string.

SQL Commands

Language: SQL Rows: 15

```

1 SELECT ENAME, SAL, NVL (COMM,0) AS Comm FROM EMP
2 WHERE EMPNO IN ((2819,2650,2654))

```

Results Explain Describe Saved SQL History

ENAME	SAL	Comm
MARTIN	1250	1400
BLAKE	2850	0
KING	5000	0

20. NULLIF (arg1, arg2)

- A. `SELECT ENAME, length (ENAME), nullif(length (ENAME),6) from EMP`

The nullif function returns null/- when both arguments are true.

Task: Replace the position of arg2 with arg1 and explore the result.

SQL Commands

Language: SQL Rows: 15

```

1 SELECT ENAME, length (ENAME), nullif(length (ENAME),5) from EMP

```

Results Explain Describe Saved SQL History

ENAME	LENGTH(ENAME)	
KING	4	4
BLAKE	5	-
CLARK	5	-
JONES	5	-

21. GROUPING FUNCTIONS

Group functions are the opposite of single-row functions.

If the input is in 10 rows then the result will be in 1 row. (Many Inputs > Single Output)

- A. `SELECT max (SAL) from EMP`
Max salary from the EMP table.
- B. `SELECT min (SAL) from EMP`
Min salary paid.
- C. `SELECT SUM (SAL) from EMP`
Total Amount paid.
Q: Show the highest-paid manager from the EMP table
- D. `SELECT avg(sal) as Avg_Sal from EMP`
The average salary of employees.
- E. `SELECT Count (*) from EMP`
Count the number of rows in a dataset with values. Replace * with a column name if required.
Q: Count the comm column from the EMP table
- F. `SELECT SUM(SAL)/Count(*) as CAL_AVG from EMP`
You can use the operators with the grouping functions.


```
SQL Commands
Language: SQL Rows: 10
1 SELECT max (SAL) from EMP
2 SELECT min (SAL) from EMP
3 SELECT SUM (SAL) from EMP
4 SELECT avg(sal) as Avg_Sal from Emp
5 SELECT Count (*) from EMP
6 SELECT SUM(SAL)/Count(*) as CAL_AVG from EMP
```

22. GROUPBY

The GROUP BY statement groups rows that have the same values into summary rows, like "find the Average Salary of each JOB". The GROUP BY statement is often used with aggregate functions (COUNT(), MAX(), MIN(), SUM(), AVG()) to group the result set by one or more columns.

A. SELECT JOB, AVG(SAL) from EMP GROUP BY JOB

Deriving the Average Salary in each JOB Profile.

Q: Find the minimum salary in each JOB Profile.

```
1 SELECT JOB, AVG(SAL) from EMP
2 GROUP BY JOB
```

JOB	
ANALYST	3000
CLERK	10375
SALESMAN	1400

23. HAVING

Group functions will not work with the where clause. Instead of where clause we will use having.

A. SELECT JOB, Count(*) from EMP GROUP BY JOB HAVING COUNT(*) = 3

Count the job profiles equal to 3.

```
4 SELECT JOB, Count(*) from EMP
5 GROUP BY JOB
6 HAVING COUNT(*) = 3
```

JOB	
MANAGER	3

24. SQL SEQUENCE

It is important to understand and remember the SQL Sequence.

- Select *
- From Table Name
- Where
- Group BY
- Having
- Order BY

25. SELECT within SELECT (Nested / Sub queries)

- A. `SELECT * FROM EMP where DEPTNO = (SELECT DEPTNO from DEPT Where LOC = 'NEW YORK')`

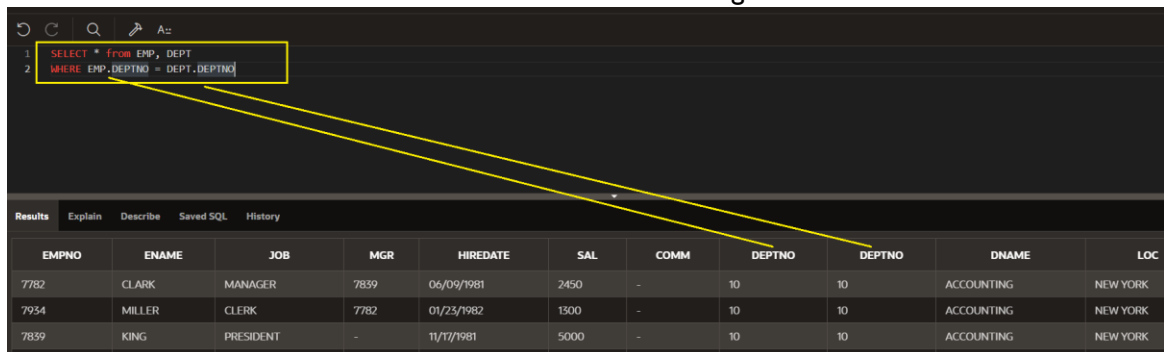
We are looking for results where DEPTNO = DEPT NO of NEW YORK. Here the () will execute first and then the global query.

```
1 SELECT * FROM EMP
2 where DEPTNO = [(SELECT DEPTNO from DEPT Where LOC = 'NEW YORK')]
```

26. JOINS

- A. `SELECT * from EMP, DEPT WHERE EMP.DEPTNO = DEPT.DEPTNO`

In the above example, we are joining two tables based on the common column which is DEPT NO. It will show all the records from EMP and DEPT tables matching the common attribute.



The screenshot shows a SQL query editor with the following query:

```
1 SELECT * from EMP, DEPT
2 WHERE EMP.DEPTNO = DEPT.DEPTNO
```

Below the query editor, the results are displayed in a table with the following columns: EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM, DEPTNO, DEPTNO, DNAME, and LOC. The results show three rows of data for employees in the Accounting department located in New York.

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	DEPTNO	DNAME	LOC
7782	CLARK	MANAGER	7839	06/09/1981	2450	-	10	10	ACCOUNTING	NEW YORK
7934	MILLER	CLERK	7782	01/23/1982	1300	-	10	10	ACCOUNTING	NEW YORK
7839	KING	PRESIDENT	-	11/17/1981	5000	-	10	10	ACCOUNTING	NEW YORK

Q: Only show the Name, Job, and Salary for New York employees.

27. INNER JOIN

- A. `select * from emp INNER JOIN dept on emp.deptno = dept.deptno`

This is the correct syntax to join two tables based on common attributes. (Only Matching Records)

28. LEFT JOIN/LEFT OUTER JOIN

- A. `select * from emp LEFT JOIN dept on emp.deptno = dept.deptno`

This will fetch all records from the right table.

Change the position of the tables like replace emp with dept and explore the result.

29. RIGHT JOIN/RIGHT OUTER JOIN

- A. `select * from emp RIGHT OUTER JOIN dept on emp.deptno = dept.deptno`
This will fetch all records from the right table.

30. FULL OUTER JOIN

- A. `select * from emp FULL OUTER JOIN dept on emp.deptno = dept.deptno`

31. EXISTS / NOT EXISTS

- A. `select * from emp Where exists (select * from emp where JOB = 'CEO')`
- B. `select * from emp Where NOT exists (select * from emp where JOB = 'CEO')`
Used with sub-queries where it returns some value.
Not an efficient way to write.

32. SELF JOINS

- A. `SELECT e.ename as EMP, m.ename as MGR from EMP e left JOIN emp m on e.mgr = m.empno`
A self-join is a regular join, but the table is joined with itself.

Q: Use above query and exclude employee don't have Manager.

33. CROSS JOINS

- A. `SELECT * from EMP CROSS JOIN DEPT`
CROSS JOINS are used to combine each row of one table with each row of another table, and return the Cartesian product of the sets of rows from the tables that are joined

34. NATURAL JOINS

- A. `SELECT * from EMP NATURAL JOIN DEPT`
A NATURAL JOIN is a JOIN operation that creates an implicit join clause for you based on the common columns in the two tables being joined.

35. USING

- A. `SELECT ENAME, JOB, LOC from EMP LEFT JOIN DEPT using (DEPTNO)`
USING Clause is used to match only one column when more than one column matches.
If the column names are different then separate them with a comma (Col1, Col2)

Q – Bring all the locations from the DEPT table.

36. EQUIJOIN

- A. `select * from emp, dept where emp.deptno = dept.deptno`
Similar to the inner join.

37. CASE WHEN

- A. `SELECT ENAME, SAL, (CASE WHEN SAL > 3000 then 'A' WHEN SAL < 1000 then '-A' else 'B' END) as Grade`
from EMP

38. OVER

- A. `SELECT D.*, count (*) over() as Total_CNT from DEPT D`

The OVER clause defines a window or user-specified set of rows within a query result set

39. PARTITION BY (Continuation of Group)

- A. `SELECT D.*, count (*) over(partition by LOC) as Total_CNT from DEPT D`

40. Create TABLE

- A. `CREATE TABLE stores (Store_ID number not null, City varchar(50))`

While creating a table make sure to have one primary column (no duplicate or null values) with the column name and data type.

41. Insert Data into TABLE

- A. `INSERT INTO stores (store_id, city) VALUES (1, 'Delhi')`
- B. `INSERT INTO stores (store_id, city) VALUES (2, 'Mumbai')`

42. Insert ALL Data into the TABLE

- A. `INSERT ALL INTO STORES (Store_id, city) VALUES (3, 'Chennai')`
`INTO STORES (Store_id, city) VALUES (4, 'Indore')`
`Select * from Dual`

While completing the INSERT ALL statement make sure to complete it with a select statement.

Remove the select statement and explore the ERROR.

43. CREATE TABLE with Primary KEY

- A. `CREATE TABLE products`
`(`
`Product_id number not null,`
`name varchar(50),`
`product_cost number (5,2),`
`store_id number not null,`

`CONSTRAINT product_pk PRIMARY KEY (Product_ID)`
`)`

To view the table properties, navigate to SQL Workshop > Object Browser > Table Name

44. INSERT ALL (Conditional Insert into Multiple Tables)

Let's create three dummy tables for the assignment dest_tbl_1, dest_tbl_2 & dest_tbl_3.

- A. `CREATE table dest_tbl_1 (id number, name varchar2 (50), hiredate date)`

- B. CREATE table dest_tbl_2 (id number, name varchar2 (50), hiredate date)
- C. CREATE table dest_tbl_3 (id number, name varchar2 (50), hiredate date)

We will now insert emp table details into the above tables. (Emp ID, Name & Hire date)

- D. INSERT ALL
INTO dest_tbl_1 (ID, name, hiredate) values (empno, ename, hiredate)
SELECT empno, ename, hiredate
from EMP

We are bringing empno, ename, and hiredate from the EMP table into dest_tbl_1.

Q: Insert the same details into the other two tables.

45. INSERT ALL with Condition

- A. INSERT ALL
When SAL <= 1500 THEN
INTO dest_tbl_1 (ID, name, hiredate) values (empno, ename, hiredate)
When SAL >1500 THEN
INTO dest_tbl_2 (ID, name, hiredate) values (empno, ename, hiredate)
ELSE
INTO dest_tbl_3 (ID, name, hiredate) values (empno, ename, hiredate)
SELECT empno, ename, hiredate, sal
From EMP

We are inserting the values based on the when a statement from the sal column in the EMP table.

46. Describe Table

- A. DESCRIBE products
To understand the table description and data type. You can click the describe icon as well.

47. ALTER TABLE

- A. ALTER TABLE products MODIFY name varchar2(50) not null
While altering more than two columns use () and separate with comma.

48. RENAME COLUMN

- A. ALTER TABLE products RENAME COLUMN name TO product_name

49. CREATE table with SELECT

- A. CREATE TABLE employees AS
SELECT empno, ename, job, sal, hiredate
FROM emp

We have extracted emp table details into employees.

50. ADDING NEW COLUMN

- A. ALTER TABLE employees
ADD store_id number

51. UPDATE VALUE to a COLUMN

- A. UPDATE employees
SET store_id = 1
WHERE JOB in ('PRESIDENT','ANALYST')

52. MODIFY COLUMN

- A. ALTER TABLE employees
MODIFY store_id number not null
We have edited the store_id column not to accept any null values.

53. MERGE STATEMENT

- A. MERGE INTO existing_customers c
USING new_customers n
ON (c.customer_id = n.customer_id)
WHEN MATCHED THEN
UPDATE SET
c.first_name = n.first_name,
c.last_name = n.last_name,
c.address_state = n.address_state,
c.email_address = n.email_address
WHEN NOT MATCHED THEN
INSERT (c.customer_id, c.first_name, c.last_name, c.address_state, c.email_address)
VALUES (n.customer_id, n.first_name, n.last_name, n.address_state, n.email_address)
Create two separate tables named existing_customers & new_customers (do minor changes in the first/last name). Run the below query to find out the difference.

54. SEQUENCE STATEMENT

- A. CREATE SEQUENCE prod_seq
MINVALUE 1
MAXVALUE 1000
START WITH 1
INCREMENT BY 1
CACHE 20 -- Not Mandatory
- B. SELECT prod_seq.NEXTVAL from DUAL

55. DELETE TABLE

- A. `DELETE FROM stores`
`WHERE store_id = 4`

56. ALTER, DROP TABLE

- A. `ALTER TABLE emp`
`DROP CONSTRAINT .....`
- B. `DROP TABLE emp`

As per the above example, we need to delete the constraint first which is available in the Object browser. Post which you can drop the entire table.

57. TRUNCATE

- A. `TRUNCATE TABLE emp`

It will remove all the data from the table without dropping the table.

58. CREATE INDEX

- A. `SELECT * FROM employees`
- B. `CREATE INDEX emp_IDX`
`ON employees (ENAME)`
- C. `SELECT * FROM employees`
`Where ename = 'KING'`

We have created index column in employees table based on the ename, which will make the search faster than the regular query.

59. ROW ID

- A. `SELECT rowid from stores`

This function will help you to identify the unique row ID defined for each row. Use this option to remove the duplicate records from the table.

60. ALL TABLES

- A. `SELECT * from all_tables`
`Where table_name = '____'`

You can view all the tables available in DB.

61. ALL TABLES/COLUMNS

- A. `SELECT * FROM all_tab_COLUMNS`

You can view all the tables with column details.

62. ALL OBJECTS

- A. `SELECT * FROM all_objects`
`WHERE object_type = 'TABLE'`

View the objects of a DB.

63. SYNONYM

- A. `CREATE SYNONYM emp_tb FOR employees`
Employees table synonym has been created as emp_tb

64. ROWNUM

- A. `SELECT rownum, ename, sal from emp_tb`
You can add the row number in each line of the table.

65. CREATE VIEW (Named Query)

- A. `CREATE VIEW ANALYST_V AS`
`SELECT * FROM employees`
`WHERE JOB = 'ANALYST'`
- B. `SELECT * FROM ANALYST_V`

66. TO VIEW the OBJECT

- A. `SELECT * FROM user_objects`
`WHERE OBJECT_TYPE = 'VIEW'`

67. TO VIEW the OBJECT based on SCHEMA OWNERS

- A. `SELECT * FROM all_objects`
`WHERE OWNER = 'WKSP_ORACLEBIMS' (--To view all put *)`
`AND object_type = 'VIEW'`

68. CREATE or REPLACE VIEW

- A. `CREATE OR REPLACE VIEW ANALYST_V AS`
Continue with the revised `SELECT` statement.

69. FETCH

- A. `SELECT * FROM employees`
`FETCH NEXT 5 ROWS ONLY`
Limiting the ROWS to 5 records only. You can use `FIRST` in place of `NEXT` as well.

70. COMMIT, SAVEPOINT & ROLLBACK

- The `COMMIT` keyword is useful for explicitly committing an insert statement. `COMMIT`.
- While inserting data into the database: Suppose you are not clear on the dataset you can create a ---
`SAVEPOINT` post your `INSERT` Comment. `SAVEPOINT my_savepoint;`
- `ROLLBACK` is similar to the undo approach. If you have not created any savepoint it will move to the last committed statement.
`ROLLBACK`
`ROLLBACK TO my_savepoint`

71. CASCADE CONSTRAINTS

This will help to drop the table and column irrespective of the primary and foreign key availability.

72. CASCADE DELETE

A foreign key with cascade delete means if a record in the parent table is deleted, then the corresponding record in the child table will automatically be deleted.

73. UNUSED COLUMN

A. `ALTER TABLE supplier`

`SET UNUSED COLUMN supplier_name`

The data for the entire column will be dropped including the constraint.

JOIN / Follow to Explore

Non-Tech to Tech is designed for aspirants, employees, and Students who don't belong/have any technical /work experience. Let's join hands & guide each other in the best possible way.

LINK TO JOIN/FOLLOW BY CLICKING THE NAME.

Thanks & Warm Regards,

[Bimal Subhasis](#)

Linkedin - [@Bimal Subhasis](#)